

Final Exam Review Questions and Answers

Based on Course Material by Zijun Yao

May 2025

Introduction

This document provides a comprehensive set of fundamental, computational, and conceptual questions and answers based on the final review material for a course at the University of Kansas, taught by Assistant Professor Zijun Yao. The questions cover Transformer Networks, Naive Bayes Classifier, K-means Clustering, Boosting, Model Generalization, and Evaluation Metrics, as specified in the review document.

1 Transformer Network

1.1 Fundamental Questions

1. **Question:** What is a token in the context of language models?
Answer: A token is a unit of text processed by language models, such as a word, subword, or punctuation mark. Tokenization converts text into these units for further processing, and it has no trainable parameters.
2. **Question:** What is the purpose of embeddings in Transformer models?
Answer: Embeddings represent tokens as vectors in a high-dimensional space, where tokens with similar meanings (e.g., "dog" and "cat") have similar vectors, capturing semantic relationships.

1.2 Computational Questions

1. **Question:** Given the text "introduction of generative ai," how many tokens might it be split into using a typical tokenizer? Refer to the document's example.
Answer: Based on the document (page 5), the phrase "introduction of generative ai" is tokenized into units. A typical tokenizer (e.g., like those used in GPT models) might split this into approximately 5 tokens: ["introduction", "of", "generative", "ai"]. The exact number depends on the tokenizer, but this is a reasonable estimate based on the document's context.
2. **Question:** If a token has an embedding vector $[1.2, -3.5, \dots, -1.9, 7.2]$, what is the dimensionality of this embedding?
Answer: The document (page 7) shows an embedding vector with components $[1.2, -3.5, \dots, -1.9, 7.2]$. The dimensionality is the number of components in the vector. Assuming a typical embedding size (e.g., 512 or 768 in Transformers), the

exact dimensionality is not specified but is typically a fixed number like 512. For this example, we assume a standard size, say 512 dimensions, as the document does not provide the exact length.

1.3 Conceptual Questions

1. **Question:** Why is positional encoding important in Transformer models?
Answer: Positional encoding (page 9) is crucial because Transformers process tokens in parallel, unlike sequential models like RNNs. Without positional encoding, the model would not know the order of tokens, which is essential for understanding the meaning of sentences. Positional encodings add information about each token's position in the sequence, enabling the model to capture word order.
2. **Question:** How does the concept of "similar embeddings for similar meanings" (e.g., dog and cat) impact the performance of a Transformer model?
Answer: Similar embeddings for tokens with similar meanings (page 7) allow Transformers to generalize better by capturing semantic relationships. For example, if "dog" and "cat" have similar vectors, the model can transfer knowledge about one to the other, improving performance on tasks like text generation or classification, especially with limited data. However, if embeddings are poorly learned, it could lead to confusion between unrelated concepts.

2 Naive Bayes Classifier

2.1 Fundamental Questions

1. **Question:** What is the main assumption of the Naive Bayes Classifier?
Answer: The Naive Bayes Classifier assumes that all features are conditionally independent given the class label, meaning the presence of one feature does not affect the others (page 41).
2. **Question:** What is Laplace Smoothing, and why is it used?
Answer: Laplace Smoothing (page 42) is a technique to handle zero probabilities in Naive Bayes by adding a small constant (usually 1) to the count of each attribute value. It prevents the entire probability from becoming zero when a feature value is absent in the training data.

2.2 Computational Questions

1. **Question:** Using the data from page 43, compute the probability $P(\text{Refund} = \text{Yes} \mid \text{Class} = \text{Yes})$ with Laplace Smoothing.
Answer: From page 43, for $\text{Class} = \text{Yes}$, the number of records is 3, and $\text{Refund} = \text{Yes}$ has 0 occurrences. With Laplace Smoothing, $P(A_i = a \mid C = c) = \frac{N_{ac} + 1}{N_c + N_i}$, where $N_{ac} = 0$, $N_c = 3$, and $N_i = 2$ (unique values: Yes, No). Thus, $P(\text{Refund} = \text{Yes} \mid \text{Class} = \text{Yes}) = \frac{0+1}{3+2} = \frac{1}{5} = 0.2$.
2. **Question:** Given the example on page 41, compute $P(Y = \text{Yes} \mid X)$ for a new instance with $\text{Refund} = \text{Yes}$, $\text{Status} = \text{Single}$, $\text{Income} = 80\text{K}$, using Laplace Smoothing for $\text{Refund} = \text{Yes}$ given $\text{Class} = \text{Yes}$.

Answer: From page 41, without smoothing, $P(\text{Refund} = \text{Yes} \mid Y = \text{Yes}) = 0$, making $P(Y = \text{Yes} \mid X) = 0$. With Laplace Smoothing (page 43), $P(\text{Refund} = \text{Yes} \mid Y = \text{Yes}) = \frac{1}{5}$. The other probabilities remain: $P(Y = \text{Yes}) = \frac{3}{10}$, $P(\text{Status} = \text{Single} \mid Y = \text{Yes}) = \frac{2}{3}$, $P(\text{Income} = 80K \mid Y = \text{Yes}) = 0.01$. Thus, $P(Y = \text{Yes} \mid X) = \frac{3}{10} \times \frac{1}{5} \times \frac{2}{3} \times 0.01 = 0.0004$.

2.3 Conceptual Questions

1. **Question:** Why does the Naive Bayes Classifier perform poorly when the independence assumption is violated?

Answer: The Naive Bayes Classifier relies on the assumption that features are independent given the class label. If features are correlated, the model overestimates or underestimates joint probabilities, leading to incorrect predictions. For example, if two features are highly dependent, the model treats them as separate, skewing the likelihood calculations (page 41).

2. **Question:** How does Laplace Smoothing affect the robustness of the Naive Bayes Classifier?

Answer: Laplace Smoothing (page 42) improves robustness by preventing zero probabilities, which can occur when a feature value is missing in the training data. This ensures the model can make predictions for unseen combinations, reducing overfitting to the training set and improving generalization, especially with small datasets.

3 K-means Clustering

3.1 Fundamental Questions

1. **Question:** What is the objective of the K-means clustering algorithm?

Answer: The objective of K-means clustering is to minimize the sum of distances (typically squared Euclidean distances) of all points to their respective cluster centroids, grouping similar objects together (page 46).

2. **Question:** What is a centroid in K-means clustering?

Answer: A centroid is the center point of a cluster, typically the mean of all points assigned to that cluster, representing the cluster's central location (page 46).

3.2 Computational Questions

1. **Question:** Given three points (1, 1), (2, 3), and (3, 2) in a cluster, compute the new centroid after one iteration.

Answer: The centroid is the mean of the points (page 46). For points (1, 1), (2, 3), (3, 2):

$$x\text{-coordinate} = \frac{1+2+3}{3} = 2,$$

$$y\text{-coordinate} = \frac{1+3+2}{3} = 2.$$

Thus, the new centroid is (2, 2).

2. **Question:** In the K-means++ algorithm (page 50), if you have selected the first centroid at point x_1 , and the squared distances from points x_2, x_3, x_4 to x_1 are 4,

9, and 16, respectively, what is the probability of selecting x_3 as the next centroid?
Answer: In K-means++ (page 50), the probability of selecting a point as the next centroid is $\frac{\min d^2(C_j, x_i)}{\sum_i \min d^2(C_j, x_i)}$. Here, the squared distances are 4, 9, and 16 for x_2, x_3, x_4 . The sum is $4 + 9 + 16 = 29$. For x_3 , the probability is $\frac{9}{29} \approx 0.3103$.

3.3 Conceptual Questions

1. **Question:** Why does the K-means algorithm's result depend on the initial choice of centroids?

Answer: K-means is sensitive to initial centroids because it converges to a local minimum of the sum of squared errors (page 48). Different starting points can lead to different cluster assignments, as the algorithm iteratively updates centroids based on the initial assignments, potentially getting stuck in suboptimal solutions.

2. **Question:** How does the K-means++ algorithm improve upon the standard K-means algorithm?

Answer: K-means++ (page 50) improves initialization by selecting initial centroids that are far apart, reducing the likelihood of poor clustering results. It chooses the first centroid randomly, then selects subsequent centroids with a probability proportional to their squared distance from existing centroids, leading to better convergence and lower sum of squared errors compared to random initialization.

4 Boosting

4.1 Fundamental Questions

1. **Question:** What is a weak learner in the context of boosting?

Answer: A weak learner is a simple classifier that performs only slightly better than random guessing (e.g., above 50% accuracy), such as naive Bayes, logistic regression, or a shallow decision tree (page 52).

2. **Question:** What is the main idea behind ensemble methods like AdaBoost?

Answer: Ensemble methods like AdaBoost combine multiple weak learners to create a strong learner by leveraging their complementary strengths, typically through weighted voting or combination of predictions (page 53).

4.2 Computational Questions

1. **Question:** In AdaBoost (page 54), if a weak classifier h_t has a weighted error rate $\epsilon_t = 0.3$, what is the weight β_t assigned to this classifier?

Answer: The weight is $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$. For $\epsilon_t = 0.3$,

$$\beta_t = \frac{1}{2} \ln \left(\frac{1-0.3}{0.3} \right) = \frac{1}{2} \ln \left(\frac{0.7}{0.3} \right) = \frac{1}{2} \ln \left(\frac{7}{3} \right) \approx \frac{1}{2} \times 0.8473 = 0.4236.$$

2. **Question:** Given a training instance (x_i, y_i) with initial weight $w_{1,i} = \frac{1}{10}$, $y_i = 1$, $h_t(x_i) = -1$, and $\beta_t = 0.4236$, compute the updated weight $w_{2,i}$ before normalization (page 54).

Answer: The weight update is $w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(x_i))$. Here, $w_{1,i} = 0.1$,

$$y_i = 1, h_t(x_i) = -1, \beta_t = 0.4236.$$

$$w_{2,i} = 0.1 \times \exp(-0.4236 \times 1 \times (-1)) = 0.1 \times \exp(0.4236) \approx 0.1 \times 1.5275 = 0.15275.$$

4.3 Conceptual Questions

1. **Question:** Why does AdaBoost focus on misclassified instances in subsequent iterations?

Answer: AdaBoost (page 54) increases the weights of misclassified instances (via the exponential update rule) to focus subsequent weak learners on harder-to-classify examples. This ensures the ensemble corrects earlier mistakes, improving overall accuracy by combining classifiers that specialize in different parts of the data.

2. **Question:** How does the weighting of classifiers in AdaBoost affect the final prediction?

Answer: In AdaBoost (page 58), classifiers with lower weighted error rates receive higher weights (β_t) in the final ensemble, meaning their predictions have more influence. The final prediction is a weighted sum of all classifiers' predictions, ensuring that more accurate classifiers contribute more to the outcome, enhancing the ensemble's performance.

5 Model Generalization

5.1 Fundamental Questions

1. **Question:** What is overfitting in the context of machine learning models?

Answer: Overfitting occurs when a model is too complex and captures noise or specific details of the training data, leading to poor performance on unseen test data (page 60).

2. **Question:** What is the purpose of regularization in linear regression?

Answer: Regularization in linear regression (page 63) controls model complexity by penalizing large weights, reducing overfitting and improving generalization to new data.

5.2 Computational Questions

1. **Question:** Given a linear regression loss function with regularization (page 63), $L(f) = \sum_{i=1}^n (y^{(i)} - f(x^{(i)}))^2 + \lambda \sum_{j=1}^n w_j^2$, if $\lambda = 0.1$ and weights $w_1 = 2, w_2 = -3$, compute the regularization term.

Answer: The regularization term is $\lambda \sum_{j=1}^n w_j^2$. For $w_1 = 2, w_2 = -3, \lambda = 0.1$:

$$\sum_{j=1}^2 w_j^2 = 2^2 + (-3)^2 = 4 + 9 = 13,$$

$$\text{Regularization term} = 0.1 \times 13 = 1.3.$$

2. **Question:** If a decision tree has a training error of 5% and a test error of 25%, is it likely underfitting or overfitting?

Answer: The large gap between low training error (5%) and high test error (25%) indicates overfitting (page 60). The model fits the training data too closely, capturing noise, and fails to generalize to the test set.

5.3 Conceptual Questions

1. **Question:** Why does overfitting lead to poor generalization in decision trees?
Answer: Overfitting in decision trees (page 62) occurs when the tree becomes too complex, with many branches that capture specific patterns or noise in the training data. This reduces its ability to generalize, as it fails to capture the underlying distribution of unseen data, leading to high test errors.
2. **Question:** How does the regularization parameter λ in linear regression balance model complexity and fit?
Answer: The regularization parameter λ (page 63) controls the trade-off between fitting the training data and keeping the model simple. A larger λ increases the penalty on large weights, encouraging simpler models that are less likely to overfit but may underfit if too large. A smaller λ allows better fit to the training data but risks overfitting if too small.

6 Evaluation Metrics

6.1 Fundamental Questions

1. **Question:** What is the formula for accuracy in a binary classification task?
Answer: Accuracy is $\frac{TP+TN}{TP+TN+FP+FN}$, where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives (page 66).
2. **Question:** What does the F-measure combine in classification evaluation?
Answer: The F-measure combines precision and recall, calculated as $F = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP+FP+FN}$ (page 67).

6.2 Computational Questions

1. **Question:** Given a confusion matrix with $TP = 50$, $TN = 40$, $FP = 10$, $FN = 20$, compute the accuracy, precision, recall, and F-measure.
Answer: From page 66-67:
 - Accuracy = $\frac{TP+TN}{TP+TN+FP+FN} = \frac{50+40}{50+40+10+20} = \frac{90}{120} = 0.75$.
 - Precision = $\frac{TP}{TP+FP} = \frac{50}{50+10} = \frac{50}{60} \approx 0.833$.
 - Recall = $\frac{TP}{TP+FN} = \frac{50}{50+20} = \frac{50}{70} \approx 0.714$.
 - F-measure = $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot 0.833 \cdot 0.714}{0.833 + 0.714} \approx \frac{1.189}{1.547} \approx 0.769$.
2. **Question:** For a classifier with $TP = 30$, $FN = 20$, $FP = 15$, $TN = 35$, compute the True Positive Rate (TPR) and False Positive Rate (FPR) for an ROC curve (page 68).
Answer: From page 68:
 - TPR = $\frac{TP}{TP+FN} = \frac{30}{30+20} = \frac{30}{50} = 0.6$.
 - FPR = $\frac{FP}{FP+TN} = \frac{15}{15+35} = \frac{15}{50} = 0.3$.

6.3 Conceptual Questions

1. **Question:** Why is the ROC curve useful for evaluating classifiers?
Answer: The ROC curve (page 68) plots TPR against FPR across different clas-

sification thresholds, showing the trade-off between correctly identifying positives and incorrectly classifying negatives. It helps evaluate a classifier's performance across all possible thresholds, making it useful for comparing models and selecting an optimal threshold based on task requirements.

2. **Question:** When might precision be more important than recall in a classification task?

Answer: Precision (page 67) is more important when false positives are costly, such as in spam email detection, where incorrectly classifying a legitimate email as spam (FP) is worse than missing some spam emails (FN). High precision ensures that positive predictions are reliable, even if some positives are missed.